

WHAT THIS SESSION IS

This class takes the chat memory from the last session and makes it permanent by storing conversations in a real database. It then shows how to write your own custom advisor, and finally turns the app multimodal: generating images, describing images, and converting speech to text and text to speech. Aim for the big picture. Exact method and property names change often between Spring AI versions.

CONTENTS

1. Quick recap and where we stopped
2. Advisors: the filter and interceptor analogy
3. Why move chat memory into a database
4. Persistent memory with JDBC (MySQL / PostgreSQL)
5. Persistent memory with MongoDB
6. Controlling how much memory is sent (tokens)
7. Writing your own custom advisor
8. The full request flow
9. Multimodal: generating an image
10. Multimodal: describing an image
11. Multimodal: speech to text and text to speech
12. Key takeaways

1 Quick recap and where we stopped

So far in Spring AI you have seen how to connect an LLM to a Spring Boot app, chat with it, call tools, use prompt templates, and give the chat a memory. That memory was **in memory**: it lived in RAM inside the JVM, so the moment the application stopped, the whole conversation was gone (RAM is volatile). We also concluded that to keep context you must pass a **conversation id**. This session picks up exactly there and moves that memory into a database.

2 Advisors: the filter and interceptor analogy

In a normal Spring Boot app, a request passes through **filters** and **interceptors** before it reaches your controller. You can rely on the default ones or add your own.

Spring AI advisors work in a similar spirit: when your prompt travels from the app to the LLM, it passes through the **advisor chain** in between. An advisor can read and change (orient) the prompt before it reaches the model, for example by attaching the old conversation history, adding a system prompt, or rewriting the request toward your own goals.

IMPORTANT DISTINCTION

Advisors are *like* filters and interceptors, but they are not the same thing. Filters and interceptors are **Spring Web MVC** components (they run before the controller). Advisors are **Spring AI** components (they run before the request reaches the LLM). Do not mix the two up.

3 Why move chat memory into a database

A quick reminder of how memory actually works, because it is easy to misunderstand. The LLM is **not** the one remembering your conversation. When you say "more", your app takes the old conversation stored on its side, attaches the new message, and sends the whole thing to the model. That is why memory costs tokens: the history is re-sent every time.

With in memory storage, that history disappears when the app restarts. To make the app truly **stateful** (surviving restarts, shareable across instances), we store the conversation in a database instead of RAM. Spring AI supports several storage backends through the `ChatMemoryRepository` abstraction.

Repository	Backend
<code>InMemoryChatMemoryRepository</code>	RAM (default, lost on restart)
<code>JdbcChatMemoryRepository</code>	Relational DB: MySQL, PostgreSQL, MariaDB, SQL Server, Oracle, HSQLDB
<code>MongoChatMemoryRepository</code>	MongoDB
<code>RedisChatMemoryRepository</code>	Redis Stack
<code>CassandraChatMemoryRepository</code> / <code>Neo4jChatMemoryRepository</code>	Cassandra / Neo4j

Reference: Spring AI Chat Memory docs, <https://docs.spring.io/spring-ai/reference/api/chat-memory.html>

4 Persistent memory with JDBC (MySQL / PostgreSQL)

This is what was demonstrated live using MySQL. PostgreSQL works identically; only the driver and connection settings differ. There is **zero difference in the Java code** between the two, only in `application.properties`.

Step 1: add the dependencies

```
<!-- Spring Data JDBC -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jdbc</artifactId>
</dependency>

<!-- Spring AI JDBC chat memory repository -->
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-model-chat-memory-repository-
jdbc</artifactId>
</dependency>

<!-- The database driver (MySQL here; use the Postgres driver for Postgres) -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
</dependency>
```

Step 2: configure application.properties

```
# Let Spring AI create its memory table automatically
spring.ai.chat.memory.repository.jdbc.initialize-schema=always
spring.ai.chat.memory.repository.jdbc.platform=mysql

# Standard datasource settings (yours will differ)
spring.datasource.url=jdbc:mysql://localhost:3306/aidb
spring.datasource.username=root
spring.datasource.password=root123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

TWO THINGS WORTH KNOWING

1. You do *not* write an `@Entity` (or a Mongo document) class. With `initialize-schema=always`, Spring AI creates its own memory table automatically on startup, with columns like `conversation_id`, `content`, `type` (user or assistant), and `timestamp`.

2. In the live class, `embedded` worked for Postgres but not for MySQL, so `always` was used. The valid values are `embedded`, `always`, and `never`.

Step 3: build the ChatClient backed by the JDBC repository

```
@RestController
@RequestMapping("/chatMemory")
public class ChatMemoryController {

    private final ChatClient chatClient;

    public ChatMemoryController(OpenAiChatModel model,
                                JdbcChatMemoryRepository
jdbcChatMemoryRepository) {

        // Memory that stores/reads messages from the database
        ChatMemory chatMemory = MessageWindowChatMemory.builder()
            .chatMemoryRepository(jdbcChatMemoryRepository)    // <-- the DB
repo
            .maxMessages(20)
            .build();

        this.chatClient = ChatClient.builder(model)
            .defaultSystem("You are a Telusko friendly AI assistant. "
                + "Keep responses helpful and remember what the
user tells you.")
            .defaultAdvisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
            .build();
    }

    @GetMapping("/chat")
    public String chat(@RequestParam String userId,
                      @RequestParam String prompt) {
        return chatClient.prompt(prompt)
            .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, userId))
            .call()
            .content();
    }
}
```

ONE EASY LINE TO MISS

Adding the dependency and the properties is not enough. You must also wire the repository into the advisor with `.chatMemoryRepository(jdbcChatMemoryRepository)`.

Without it, the memory would still use the default in memory store. This exact line was the one forgotten (and then fixed) live in class.

What you see when it runs

- Ask about a topic with `userId=4`, then say "tell me more about his YouTube channel", and it recalls the earlier context, because both your messages and the assistant replies are stored under that id.
- Switch to `userId=6` and the model has no memory of user 4's conversation. Each id is a separate chat room.
- Stop and restart the app: the conversation is still in the database (unlike in memory).
- It still cannot answer about events after its training cutoff. For current or private data you use RAG or tool calling (covered in later sessions).

5 Persistent memory with MongoDB

A common question in class was: "must it be a SQL database, or can I use MongoDB?" The answer is yes, MongoDB works too. Since MongoDB is a NoSQL store, the repository component changes, but the idea and most of the code stay the same. Here are the exact steps using Spring AI's official MongoDB chat memory repository.

Step 1: add the dependencies

```
<!-- Spring Data MongoDB -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

<!-- Spring AI MongoDB chat memory repository -->
<dependency>
    <groupId>org.springframework.ai</groupId>
    <artifactId>spring-ai-starter-model-chat-memory-repository-
mongodb</artifactId>
</dependency>
```

Step 2: configure application.properties

```
# Connect to your MongoDB (local or Atlas)
spring.data.mongodb.uri=mongodb://localhost:27017/aidb
# or, instead of a uri:
# spring.data.mongodb.host=localhost
# spring.data.mongodb.port=27017
# spring.data.mongodb.database=aidb

# Optional Spring AI MongoDB memory settings
spring.ai.chat.memory.repository.mongo.create-indices=true    # auto-create
indices on startup (default false)
spring.ai.chat.memory.repository.mongo.ttl=0                  # time-to-live in
seconds, 0 = keep forever
```

As with JDBC, you do not write a document class yourself. Spring AI stores the conversation in its own collection.

Step 3: use the MongoDB repository (only one line changes)

```
public ChatMemoryController(OpenAiChatModel model,
                           MongoChatMemoryRepository
                           mongoChatMemoryRepository) {

    ChatMemory chatMemory = MessageWindowChatMemory.builder()
        .chatMemoryRepository(mongoChatMemoryRepository)    // <-- Mongo
instead of JDBC
        .maxMessages(20)
        .build();

    this.chatClient = ChatClient.builder(model)
        .defaultSystem("You are a Telusko friendly AI assistant.")

        .defaultAdvisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
        .build();
}
```

The controller method (with `ChatMemory.CONVERSATION_ID`) is exactly the same as the JDBC version. Swapping databases is just swapping the injected repository.

ONE CAVEAT WITH MONGODB (FROM THE OFFICIAL DOCS)

The MongoDB repository does not store tool call messages: assistant messages that contain tool calls, and tool response messages, are filtered out. For plain chat memory this is fine. If you rely heavily on tool calling and also need those exact messages persisted, prefer a store that keeps them, or handle it in your own logic. If your conversation tables grow large over time, you can also cap them (for example, delete old messages after a threshold, or use the Mongo `ttl` to auto-expire).

6 Controlling how much memory is sent (tokens)

`MessageWindowChatMemory` is a **sliding window**. Even if the database holds 40 or 50 messages for a conversation, only the last `maxMessages` are attached to the new prompt and sent to the model.

```
MessageWindowChatMemory.builder()  
    .chatMemoryRepository(repository)  
    .maxMessages(10)    // only the last 10 messages travel to the model  
    .build();
```

WHY THIS MATTERS

You pay per token, and that includes the history you re-send. Sending the full conversation every time would blow up the cost and can hit the model's context window limit. More remembered messages means more tokens, higher cost, and slower responses. Pick a window that fits your use case; more is not always better.

7 Writing your own custom advisor

Chat memory used a built in advisor. You can also write your own. A custom advisor lets you rewrite the user's prompt toward your own goals before it reaches the model. You implement the `CallAdvisor` interface.

```

public class TutorStyleAdvisor implements CallAdvisor {

    @Override
    public String getName() {
        return "TutorStyleAdvisor";
    }

    @Override
    public int getOrder() {
        return 0;    // position in the advisor chain
    }

    @Override
    public ChatClientResponse adviseCall(ChatClientRequest request,
    CallAdvisorChain chain) {

        // 1. Get the original user prompt from the request
        String originalPrompt = request.prompt().getContents();

        // 2. Modify it to add tutorial-style instructions
        String modifiedPrompt = originalPrompt
            + "\n\nPlease answer in a tutorial style, with clear explanations "
            + "and an example where appropriate.";

        // 3. Build a new request carrying the modified prompt
        Prompt newPrompt = new Prompt(modifiedPrompt);
        ChatClientRequest newRequest =
request.mutate().prompt(newPrompt).build();

        // 4. Pass it down the chain to the model
        return chain.nextCall(newRequest);
    }
}

```

Register it on the ChatClient alongside the default advisors:

```

this.chatClient = ChatClient.builder(model)
    .defaultAdvisors(
        MessageChatMemoryAdvisor.builder(chatMemory).build(), // built in
        new TutorStyleAdvisor()                               // your
    custom one
    )
    .build();

```

ADVISOR VERSUS SYSTEM PROMPT VERSUS PROMPT TEMPLATE

They feel similar but differ. A **system prompt** only sets the tone/role and is optional. A **prompt template** builds the whole prompt from a few user keywords. A **custom advisor** sits in the chain and can rewrite *any* request programmatically, chain with other advisors, and apply across many endpoints. In the demo, the same question answered through the custom advisor gave a noticeably richer, better organised response than the system prompt alone. Advisors are also not limited to memory or rewriting; you will see them used again for RAG (a question answer advisor) in later sessions.

8 The full request flow

```

Client (prompt)
  -> Spring Boot app
    -> ChatClient
    -> Advisor chain (system prompt, prompt template, memory, custom
advisors...)
    -> Chat memory repository (JDBC / MongoDB / Redis...) stores & reads
history
    -> AI model (any provider)
    -> response back to the client

```

Everything you have added (tools, prompt templates, system prompts, custom advisors, and now persistent memory) happens on your side, before and around the call to the model. The code stays provider agnostic: nowhere is OpenAI, Gemini, or Anthropic hard coded, so the same code works with any model.

9 Multimodal: generating an image

Spring AI can do more than text. First goal: give text, get an image back. For this you use an image model (here OpenAI's image model) in addition to the chat client.

```
@RestController
@RequestMapping("/image")
public class ImageController {

    private final OpenAiImageModel imageModel;

    public ImageController(OpenAiImageModel imageModel) {
        this.imageModel = imageModel;
    }

    @GetMapping("/generate/{prompt}")
    public String generateImage(@PathVariable String prompt) throws IOException
    {

        // Optional: customise size, quality, style, model
        OpenAiImageOptions options = OpenAiImageOptions.builder()
            .quality("hd")
            .height(1024).width(1024)
            .build();

        ImagePrompt imagePrompt = new ImagePrompt(prompt, options);
        ImageResponse imageResponse = imageModel.call(imagePrompt);

        // Two ways to get the result: a URL, or base64 bytes you decode and
        save
        String b64 = imageResponse.getResult().getOutput().getB64Json();
        byte[] imageBytes = Base64.getDecoder().decode(b64);
        Files.write(Paths.get("ai-generated-image.png"), imageBytes);

        return "Image generated successfully";
    }
}
```

Put the model choice in `application.properties` rather than hard coding it:

```
spring.ai.openai.image.options.model=gpt-image-1
```

REAL CLASSROOM GOTCHAS

A couple of live hiccups worth remembering: (1) generated images can be large, so if the response comes back null you may need to raise the servlet request/response size limits. (2) The model enforces a **safety system**: some prompts get rejected ("rejected by the safety system") and it will refuse to generate that image. Change the prompt and try again. Note also that, unlike text calls, the image response does not expose a token usage count, so you can only estimate cost.

10 Multimodal: describing an image

The reverse direction: upload an image plus a text question, and the model describes it. You attach the uploaded file to the prompt as media. This needs a model that supports multimodal input.

```
@PostMapping("/describe")
public String describeImage(@RequestParam String prompt,
                             @RequestParam MultipartFile file) {

    return chatClient.prompt()
        .user(u -> u.text(prompt)
            .media(MimeTypeUtils.IMAGE_JPEG, file.getResource()))
    // attach image
        .call()
        .content();
}
```

The key line is attaching the media to the user message. Sending only the text ("describe this image") without the attached image would leave the model with nothing to look at. Spring AI sends the text and the image together in one call. In the demo, uploading a building photo returned "a modern high-rise building with a sleek design", and a desert photo returned "a vast desert landscape with rolling sand dunes".

11 Multimodal: speech to text and text to speech

Speech to text (transcription)

Upload an audio file, get the transcript back, using the OpenAI transcription (audio) model.

```
private final OpenAiAudioTranscriptionModel audioModel; // injected

@PostMapping("/speechToText")
public String speechToText(@RequestParam MultipartFile file) {
    AudioTranscriptionPrompt prompt =
        new AudioTranscriptionPrompt(file.getResource());
    return audioModel.call(prompt).getResult().getOutput();
}
```

You cannot control a "voice" here (there is no voice in transcription), but you can control things like the output language and response format.

Text to speech

Give text, get audio bytes back, using the OpenAI speech model with options for voice and speed.

```
private final OpenAiAudioSpeechModel speechModel; // injected

@GetMapping("/textToSpeech")
public byte[] textToSpeech(@RequestParam String prompt) {

    OpenAiAudioSpeechOptions options = OpenAiAudioSpeechOptions.builder()
        .speed(1.5f) // 0.25
        (slowest) to 4.0 (fastest)
        .voice(OpenAiAudioApi.SpeechRequest.Voice.FABLE) // pick a voice
        .build();

    SpeechPrompt speechPrompt = new SpeechPrompt(prompt, options);
    return speechModel.call(speechPrompt).getResult().getOutput(); // audio
    bytes (e.g. save as .mp3)
}
```

A USEFUL TRICK SHOWN IN CLASS

The model will not translate an English audio into, say, Kannada in a single step. But you are in control of the flow: take the transcript the model returns, then make a second chat call ("translate the following to Kannada: ...") and return that to the user. You are not fully dependent on the model in one shot; you can chain calls and customise the result. Voice, speed, language, and format can all be set either in code (options) or in `application.properties`.

12 Key takeaways

- In memory chat is lost on restart. Store it in a database via a **ChatMemoryRepository** to make the app stateful.
- **JDBC** (MySQL/Postgres) and **MongoDB** both work. Swapping databases changes only the dependency, a few properties, and the injected repository; the Java logic stays the same. Spring AI creates its own table/collection, so no entity or document class is needed.
- Always wire the repository into the memory with `.chatMemoryRepository(...)`, and remember MongoDB filters out tool call messages.
- Use `maxMessages` (sliding window) to control how much history is sent, which controls token cost.
- **Custom advisors** (implementing `CallAdvisor`) let you rewrite prompts in the chain, more powerful than a plain system prompt.
- Spring AI is **multimodal**: generate images, describe images, transcribe speech, and synthesise speech, all provider agnostic.
- Model safety may reject some image prompts; image responses do not report token usage.

COMING NEXT

Controlling the output (structured output and limiting tokens), then the big topics: embeddings and vector stores (PGVector, Redis, run via Docker), RAG and advanced RAG, and MCP. This was class five of the course.

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO

TELUSKO